

Maximizing Single-Node Throughput for 8B GPT Pretraining on Grace-Hopper GH200

Sarah Kuhn, Robert Paiz, Mykhaylo Gershma, Maximiliaan van der Hart

Abstract

Training throughput on GPU clusters is a primary constraint on the scale of language-model pretraining: Under fixed hardware and a fixed dataset, every gain in tokens/second/GPU translates directly into either more training tokens within a fixed wall-clock budget or cost savings at a fixed token target. This work addresses tokens/second/GPU maximization for a prescribed model scale and parallelism tier, tackling the Single-GPU tier of the Gipfelsturm Challenge 2 on the Alps Clariden supercomputer. We target an 8B-parameter dense GPT trained on a single node of four GH200 Grace-Hopper chips in a pure data-parallel configuration, with no model parallelism. This setting eliminates inter-node communication and enables isolation of the interaction between 8-bit precision, memory footprint, and per-GPU micro-batch size, axes that remain important when scaling across parallelism tiers. We provide a training configuration achieving a **+48.6% throughput improvement** over the Megatron-LM BF16 baseline. The gain is not attributable to any single axis: it emerges from the joint tuning of FP8 precision, activation memory, and micro-batch size, and would be missed by a greedy single-factor search.

Introduction

Training throughput bounds how far we can push language-model pretraining: faster training means either more tokens within a fixed budget or the same tokens at lower cost. Many engineering choices feed into this throughput: numerical precision, micro-batch size, memory layout, intra- and inter-node parallelism, interconnect bandwidth, and kernel selection.

We restrict attention to a single node and to main axes: reduced precision, memory recovery, and micro-batch size. *Reduced precision* swaps BF16 or FP32 operands in matrix multiplies for narrower formats that the tensor cores can process faster. *Memory recovery* refers to changes that shrink peak activation memory e.g. fused kernels and recomputation. *Micro-batch size* (MBS) is how many sequences each GPU processes per forward-backward pass before gradient accumulation. These axes are often studied in isolation: FP8 should accelerate matrix multiplies, larger micro-batches improve hardware utilization, and memory optimizations are treated as an orthogonal concern. We find that in our setting this separation can hide the configuration that actually maximizes throughput.

For 8B GPT pretraining on a single 4×GH200 node, the maximum we measure using the Megatron-LM framework is **16,096 tok/s/GPU**, a **+48.6% improvement** over the unmodified Megatron-LM BF16 baseline of 10,829 tok/s/GPU. The maximum is the composition of three steps: (i) FP8 hybrid arithmetic, which gives +40.9% by itself; (ii) a Transformer-Engine-fused cross-entropy that is throughput-neutral but frees ~1.6 GB of activation memory; and (iii) doubling the micro-batch from 1 to 2, which adds another +6.7% but only because step (ii) freed the memory it needs. Step (ii) is invisible to greedy search because it produces no throughput gain on its own. However, we found that without it, the throughput optimized doubled micro-batch out-of-memories. The optimum therefore sits on a joint configuration path that no single factor tuning can find.

Our contributions are:

- (i) a configuration that measures **16,096 tok/s/GPU** on the Challenge 2 Single-GPU tier;
- (ii) an explanation of the memory–throughput coupling that links FP8 arithmetic, the FP32 logits buffer of the cross-entropy implementation and MBS, and which leads to our 16,096 tok/s/GPU configuration.
- (iii) an overview over negative-result covering attempted optimizations across the precision, parallelism, execution, memory-recovery, and attention-backend axes including ablations.

We start providing Background on Alps/Clariden hardware and the Megatron-LM and Transformer-Engine software focusing on the fraction we use, and introduce FP8. Methodology states the model, hardware, dataset, and how we measure. Optimization Path walks from baseline to our final training configuration. In Mechanism why aim to explain why each step has the effect it does. *§Negative Results/Ablations* lists the twelve negative results which we categorized. Optimization Path, Ablations and Discussion form our evaluation. ***

Background

We summarize the hardware and software stack we train on, and position our work against prior FP8 results.

Grace Hopper GH200 and Alps Clariden. Training runs on Alps Clariden nodes, each containing four GH200 modules. Every module pairs a Grace CPU with a Hopper GPU and its high-bandwidth memory. The CPU–GPU link is cache-coherent, so both sides dereference the same addresses without an explicit copy. Intra-node traffic uses NVLink, inter-node traffic uses Slingshot-11 at an order of magnitude below the intra-node bandwidth. In our setup the only inter-rank traffic is the data-parallel gradient all-reduce, which stays inside the node and uses NVLink.

Reduced precision on Hopper tensor cores. One axis of our tuning is precision. Hopper tensor cores support TF32, BF16, FP16, and two 8-bit formats with different range–precision trade-offs (E4M3 and

E5M2). Each halving of precision roughly doubles peak throughput since more FLOPS on less bits are performed. We use hybrid FP8, E4M3 in the forward pass (more precision), E5M2 in the backward pass (wider range). Stability comes from per-tensor dynamic scaling and Megatron allows us to tune when the update of the maximal value to quantize the tensor happens e.g. the delayed flag amortizes updates over a history instead of picking the most current value each time.

Megatron-LM and Transformer Engine. The training framework provided in the challenge and used for our training is the Megatron-LM [Shoeybi et al. 2019, Korthikanti et al. 2022]. Different flags give access to preimplemented features for distributed training: A distributed optimizer shards optimizer state across data-parallel ranks (ZeRO stage 1) and the flags like `--overlap-grad-reduce / --overlap-param-gather` allow to overlap computations to hide communication. Relevant to our work is that NVIDIA’s Transformer Engine provides FP8 implementations of matmul, attention, and normalization kernels, and there is a flag to route GEMMs to the FP8 tensor cores. We access the existing FP8 set up via Megatron via the `--transformer-impl transformer_engine` flag.

Memory in 8B GPT training. Another tuning axes is memory. Our 8B model uses the Grace-Hopper high bandwidth memory for parameters and gradients (less in FP8 than BF16), the sharded optimizer state and the activation history for the backward pass. We learned that additionally short-lived buffers are kept e.g. for the cross-entropy logits. Megatron’s default cross-entropy builds a logits tensor before reducing it to the loss which can contribute to a worst-case memory use, especially when it coincides with the activation memory peaks. To mitigate we found the Transformer Engine flag `--cross-entropy-fusion-impl te`, which computes the loss in tiles and never materializes the full tensor. In §*Mechanism* we show that this buffer is what determines the largest micro-batch that fits in our setting.

Related work on FP8 training. FP8 mixed-precision training for transformers was first characterized by NVIDIA [Micikevicius et al., 2022], who demonstrated end-to-end FP8 GPT training with throughput gains across model scales. DeepSeek described an end-to-end FP8 pipeline for a Mixture-of-Experts model. We build on the fact the FP8 can improve throughput and focus on which joint configurations with FP8 contribute to maximizing throughput.

Methodology

We describe the model, hardware, dataset, and the measurement we used also to discard high-variance runs. We never run full training, following gipfelsturm, we probe 50 steps, which is sufficient to sample throughput.

Model, Dataset, Tokenizer. Dense, decoder-only GPT-style transformer with 32 layers, Megatron’s default. Sequence length 4096, vocabulary 50,257 (GPT-2 BPE), 8.0 B parameters. Linear biases are disabled and the fused attention kernel applies the mask, so the data loader does not

materialize attention masks. As defined in our task, we use the Nemotron-ClimbMix dataset (climbmix_small_megatron) with the GPT-2 BPE tokenizer (50,257 vocab).

Hardware and partition. Single node of Alps Clariden with the alps3 container image. Our Slurm flags: `--gpus-per-node=4 --ntasks-per-node=1 --cpus-per-task=288 --mem=460000`. We used the normal partition for some runs and the debug partition for some 50-step throughput runs, which schedules in about a minute and gives the lowest measurement variance.

Training configuration. Each run is 50 iterations at global batch size 256 and micro-batch size 1 or 2. Optimizer: Adam ($\beta_1=0.9$, $\beta_2=0.95$, weight decay 0.1, gradient clipping 1.0). Learning rate 3×10^{-4} with a 10-iteration warmup and constant thereafter. The precision-aware optimizer (`--use-precision-aware-optimizer`) keeps BF16 gradients instead of FP32, without it, we observed that FP8 runs OOM. Selective activation recomputation is on everywhere.

Distributed training features. We hold TP=PP=1 throughout, as required by the Single-GPU tier. The four GPUs run as data-parallel ranks with the distributed optimizer (ZeRO-1), overlapping gradient reduce and parameter gather.

Measurement methodology. We log throughput per iteration as tok/s/GPU via Megatron’s `--log-throughput`. For each run we report `mean_last_10`, the mean over iterations 41–50, discarding the first 40 as warm-up. To verify that the run reached a steady state, we also compute `mean_all` over all 50 iterations and define

$$v_{\text{run}} = \frac{|\text{mean}_{\text{all}} - \text{mean}_{\text{last 10}}|}{\text{mean}_{\text{last 10}}}$$

rejecting any run with $v_{\text{run}} > 5\%$. We observe that runs on the debug partition hold $v_{\text{run}} \in [0.5\%, 1.5\%]$ while some runs on the normal partition are subject to cluster contention and can reach 15–25%.

Peak memory is read from `torch.cuda.max_memory_reserved()` at the end of each run. TFLOP/s/GPU is derived from tok/s/GPU using Megatron’s per-token FLOP estimate $6N + 12LH^2S$.

Metric	Definition
Throughput	<code>mean_last_10</code> over iterations 41–50
Steady-state check	$v_{\text{run}} = \text{mean}_{\text{all}} - \text{mean}_{\text{last 10}} / \text{mean}_{\text{last 10}}$
Rejection threshold	$v_{\text{run}} > 5\%$
Peak memory	<code>torch.cuda.max_memory_reserved()</code>

Metric	Definition
Compute	$\text{tok/s/GPU} \times (6N + 12LH^2S)$

Optimization Path

We describe the four configurations on the path to our maximal-throughput setup (Figure 1), each adding one change to the previous. We explicitly exclude failed attempts here and defer them to a later section.

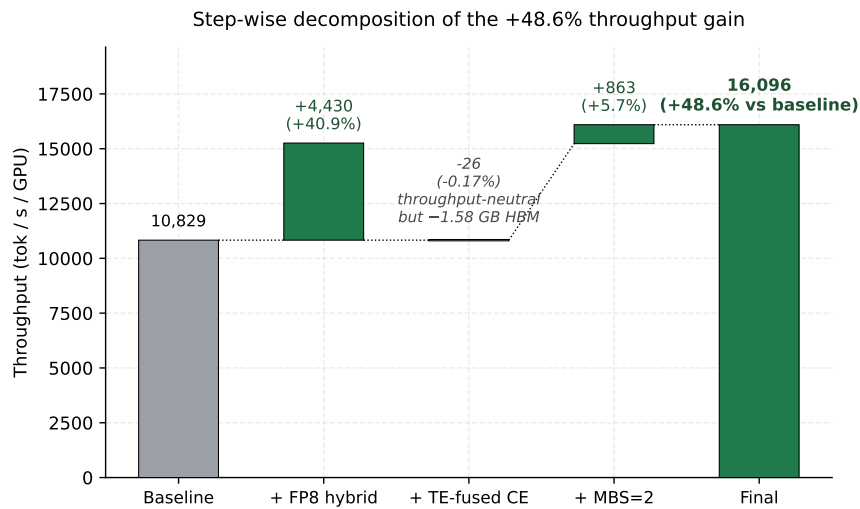


Figure 1: Per-step contribution to tok/s/GPU. Throughput-neutral optimization on their own can unlock subsequent throughput boosts.

Run 1 — BF16 baseline. The unmodified Megatron launcher runs BF16 mixed precision at MBS=2 and GBS=256. Our measured throughput is **10,829 tok/s/GPU**, matching the published reference (~10,882) (within noise). We used this to 1.) confirm that our measurement is sound 2.) have a baseline against which we can report every gain/loss.

Run 2 — Add FP8 hybrid arithmetic (+40.9%). Adding `--transformer-impl transformer_engine`, `--fp8-format hybrid`, delayed scaling, and `--use-precision-aware-optimizer` to route every GEMM in the transformer to the FP8 tensor cores. Throughput rises to **15,259 tok/s/GPU (+40.9%)**: compute goes from ~503 to ~707 TFLOP/s/GPU, time falls from 24.2 s to 17.2 s, and peak HBM drops from 89.3 GB to 77.7 GB. To make this configuration run we also

needed to add memory fixes such as the explicit `--bf16` and cut micro-batch from 2 to 1, to have smaller logits buffers which pushed peak HBM over the 96 GB cap. However, this started drawing our attention toward logits buffers and memory optimizations that could unlock larger MSB.

Run 3 — Add Transformer-Engine-fused cross-entropy. We added `--cross-entropy-fusion-impl` to swap the default cross-entropy for a chunked Transformer Engine that never builds the full logits tensor. Throughput is **15,233 tok/s/GPU**, a **-0.17%** move from Run 2. Peak HBM drops by **1.58 GB** (from 77.23 to 75.65 GB). Hence this step seems throughput-neutral but is memory-recovering. On its own it would be dropped by greedy search for throughput.

Run 4 — Raise micro-batch to 2. After removing the FP32 logits buffer we now made the MBS=2 fit in HBM. This configuration reaches **16,096 tok/s/GPU** at 746 TFLOP/s/GPU and 16.29 s/iteration, **+6.7% over Run 3** and **+48.6% over the BF16 baseline**. We observed a run variance of ~0.5% and a $\text{lm loss} = 7.29$ at iteration 50 which matches the baseline in loss. Without Run 3's cross-entropy fusion, this run would OOM. (W&B run 3 rtoxm4k)

Figure 2 below plots throughput together with peak HBM and Table 2 summarizes the per-run numbers.

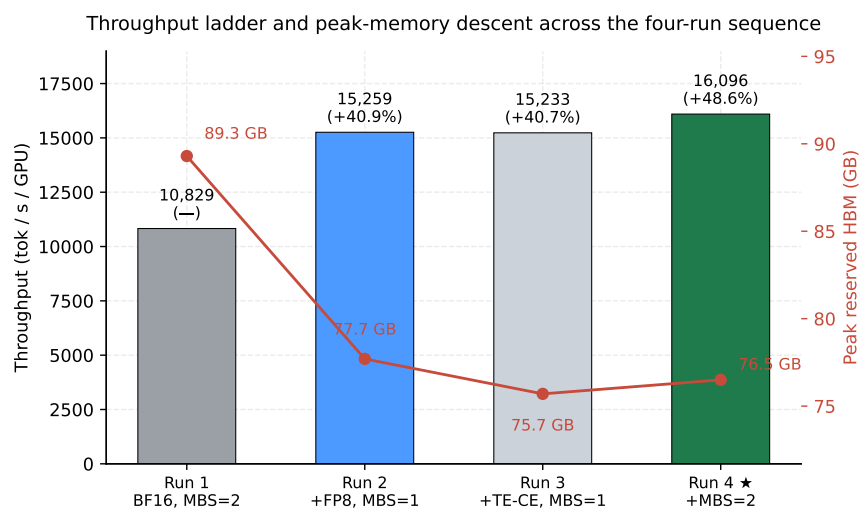


Figure 2: Throughput (bars, left axis) and peak reserved HBM (line, right axis) across the four runs. Throughput climbs while peak HBM descends before it rises slightly in Run 4 given the doubled micro-batch.

Table 2. Per-run metrics from baseline to final configuration. Values are derived as written in our methodology. Any Δ is against the BF16 baseline.

Stage	Configuration	tok/s/GPU	TFLOP/s/GPU	Iter (s)	Peak mem	Δ vs baseline
					(GB)	
Run 1	BF16, MBS=2	10,829	503	24.20	89.3	—
Run 2	+ FP8 hybrid, MBS=1	15,259	707	17.18	77.7	+40.9%
Run 3	+ TE-fused CE, MBS=1	15,233	706	17.21	75.7	+40.7%
Run 4 ☒	+ MBS=2	16,096	746	16.29	76.5	+48.6%

Mechanism: Memory-Throughput Coupling

After we described what each step did, we try to explain why and how the steps lead to our final configuration.

FP8 Precision. FP8 tensor cores deliver 2× the peak FLOPs of BF16 on the same CPU-GPU module as halving the bit width lets twice as many MMAs to be performed (just on less bits). Naturally a number close to 2x might be expected but since not all operations are run in FP8 (some stay in FP16 per implementation in Megatron), the speed up is limited by the non-GEMM fraction of a step.

Avoiding the FP32 logits buffer. Megatron’s default cross-entropy materializes an FP32 $[\text{seq} \times \text{MBS} \times \text{vocab}]$ tensor before reducing to the loss which has size

$$S_{\text{logits}} = \text{seq} \times \text{MBS} \times \text{vocab} \times 4 \text{ bytes}$$

. In our case at $\text{seq} = 4096$, $\text{vocab} = 50,257$ this yields **0.82 GB per unit of micro-batch**. If the buffer colides and peaks together with the activation memory peak, we reached the HBM limit and OOM. Via the fused kernel we can compute the loss in tiles and not build the full tensor, which recovers 1.58 GB and fits MBS=2.

MSB translates to Matrix dimension. Doubling the MBS doubles a dimension in matrix of the FFN forward matrix multiply. Processing a larger batch then amortizes various scheduling overheads across more output rows e.g. reusing each shared-memory tile of the weight across more rows. By increasing the MSB from 1 to 2 allowed us to move more into the compute-bound regime, from which we claim the +6.7% comes from.

The Benefit Of Coupling. What we learned is that two directly throughput positive axes are linked by a buffer that seems throughput neutral. TE-fused CE shows no throughput gain on its own, but recovers exactly the HBM that the doubled micro-batch needs.

Negative Trials and Ablations: Runs Across the Search Space

Beyond the final “winning” configuration we ran other trials e.g. even higher MSB. Each one was either measured with lower throughput than our final or directly rejected at run startup because the flags were incompatible. We group them by category based on the axes it affects (Parallelism, Execution, Memory, Precision) and show full numbers in Table 3. Each ablation is against the final configuration.

Precision recipe and format. Switching from the delayed recipe to tensorwise costs **-4.56%**: delayed amortizes amax updates across a 1,024-step history, while tensorwise recomputes the scale on every call, and the per-step cost seems to outweigh any precision benefit. Switching the FP8 format from hybrid to plain e4m3 costs -1.48%. The flag `--first-last-layers-bf16`, which would keep the first and last layers in BF16, is incompatible with the delayed and failed at startup.

Parallelism. We tried tensor parallelism with sequence parallelism at TP=2 and MBS=4 which cost -15.1%. We observed that without `--tp-comm-overlap`, the TP all-reduces are serialized and add ~3.1 s/iteration (would anyways be not part of the single node tier which requires TP=1) The flag `--fp8-param-gather`, which would gather FP8-cast parameters before each GEMM, is incompatible with `--use-precision-aware-optimizer`.

Execution and graph capture. TO DO didnt understand it at all

Memory recovery at higher micro-batch. Three attempts to go past MBS=2 all OOM. MBS=3 failed (the logits buffer is 2.46 GB and activations are also higher). Adding selective recompute saved only ~1-3 GB which wasn’t enough. All three of our MBS=4 also failed and no combination of recompute and offload reclaimed enough memory. We concluded: MBS=1 fits (15,086 tok/s/GPU), MBS=2 is optimal (16,096), MBS=3 OOM, MBS=4 OOM.

Attention backend. We onced forced the FlashAttention library via `--attention-backend flash` which cost **-13.84%**. We assume that the default route autmaticall picks the Hopper-optimal for our shape (seq = 4096, FP8) which is why the external FlashAttention path is slower on our exact workload.

Table 3. Ablations against the final config, by category. Δ is the percent change in mean_last_10 tok/s/GPU from the 16,096. No meassurement means failed or OOM.

Class	Change	tok/s/GPU	Δ vs champion	Mechanism
Precision	recipe tensorwise (vs delayed)	14,625	−4.56%	per-step amax recompute outweighs accuracy gain
Precision	format e4m3 (vs hybrid)	15,858	−1.48%	E4M3 truncates long-tailed gradient distribution
Precision	--first- last- layers- bf16	—	FAILED	incompatible with delayed recipe
Parallelism	TP=2 + SP + MBS=4	12,933	−15.1%	TP all-reduces serialized without --tp-comm- overlap; off-spec
Parallelism	--fp8- param- gather	—	FAILED	FP8 Adam kernel expects Int16 master; precision- aware-opt is Float32
Execution	CUDA graphs (+ expand- able_segments)	14,982	−1.44%	NCCL graph reg forced off; Accu- mulateGrad stream mismatch
Execution	CUDA graphs (no allocator workaround)	14,223	−6.6%	fragmentation costs more than graph capture saves

Class	Change	tok/s/GPU	Δ vs champion	Mechanism
Execution	--ddp- bucket- size 100M	15,201	+0.76% (noise)	within run-to-run variance, not a real gain
Memory	MBS=3 (bare and + recompute)	—	OOM	logits buffer 2.46 GB; recompute insufficient
Memory	MBS=4 (bare, + recompute, + CPU offload)	—	OOM (×3)	~2× activations vs MBS=2; cannot be reclaimed
Memory	drop --use- precision- aware- optimizer	—	FAILED	FP32 master grads OOM the optimizer state
Attention	-- attention- backend flash (vs auto)	13,869	-13.84%	TE-auto picks cuDNN-fused (Hopper- optimal for our shape)

Few words on Loss/ Convergence

Our challenge tackles throughput, not convergence nor loss. However, we still logged per-iteration ℓ_m loss during the 50-iteration window as a sanity check also with respect to model utility. We ensured that no FP8 step, as described in the optimization path section, shows numerical instability over the 50 steps. The baseline and final configuration shows the same loss convergence curve over the 50 steps. (hopefully, FIGURE)

TO DO : Plot here that shows the loss curve of the baseline and the final cnofgi to make the point that over the 50 steps convergence seems to be stable (is to short to say over full horizon but at least over 50 steps is numerically as stable as baseline)

Issues and Work-Arounds

In this section we describe some issue we encounter during our work and the work arounds we employed.

Contention episode During our work, four days of repeated measurements of the identical final configuration gave systematically lower throughput with much higher run variance (while loss trajectories still matched the baseline at every iteration). Then after a few days it was resolved and we assume there was contention on Slingshot during that time period(?).

Date	W&B run	mean_last_10 (tok/s/GPU)	v_{run}	Verdict
2026-05-04	6wxov536	15,324	~1%	✓ reference
2026-05-15	0nus26mz	11,444	~25%	✗ contention
2026-05-15	9m7o4ytz	12,672	~17%	✗ contention
2026-05-17	0ytlvleya	13,149	~18%	✗ contention
2026-05-18	28e9vrxi	14,954	~1%	✓ resolved

Leveraging the debug partition. Since the production partition is fair-share throttled, we often had to wait long making it hard to work with. Thus we fallback onto the debug partition in certain cases which schedules in about one minute and consistently produces $v_{\text{run}} \approx 0.005$. We argued that it might be the right tool for very short 50-step throughput probes that we performed (not for full training).

TO DO -> ADD MORE IF NEEDED

Discussion

The memory-throughput coupling argument we found/use applies to any Hopper-class workload where some throughput-neutral intervention can free enough HBM to enable throughput optimizations such as larger MSB. The specific buffer in our case (the FP32 logits tensor scaling with $\text{seq} \times \text{MBS} \times \text{vocab} \times 4$) is workload-specific, but the argument that joint tuning of memory and compute axes can reach configurations greedy search cannot is general. Optimizing for tok/s/GPU includes looking for throughput-neutral memory recoveries that unlock new optimizations rather than discarding them.

Our +48.6% number is specific to the 8B / TP=PP=1 / 4xGH200 tier and we claim that at larger model scales tensor parallelism becomes required. Once $\text{TP} > 1$, the dominant overhead is intra-node communication.

A limitation that our project has is that the throughput is measured over 50 iterations not throughput averaged over a full training run (which would include the warmup and any longer-run FP8 stability effects). Second, an interesting measure to report would be tok/s/\$. Not only tok/s/GPU. Also other model architectures would be interesting e.g Mixture-of-Experts routing (tried but failed...) and ZeRO stages 2 and 3.

Conclusion

Our findings on the Single-GPU tier of *Gipfelsturm* Challenge 2:

- **Throughput maximum: 16,096 tok/s/GPU**, a +48.6% improvement over the Megatron-LM BF16 baseline of 10,829 tok/s/GPU. The final configuration is FP8 hybrid arithmetic with delayed scaling, Transformer-Engine-fused cross-entropy, and micro-batch size 2 (W&B run 3r to xm4k).
- **The path is non-greedy.** FP8 alone gives +40.9%, doubling the micro-batch adds +6.7%. The two compose only because a third step, Transformer-Engine-fused cross-entropy, throughput-neutral on its own, frees the 1.6 GB FP32 logits buffer that would otherwise OOM the doubled micro-batch. Under single-step search the memory recovery looks worthless but in our case it is decisive.
- **Twelve further interventions were measured or rejected**, bounding the search along the precision, parallelism, execution, memory-recovery, and attention-backend axes. We find that no further throughput is reachable for our tier without breaking the TP=PP=1 rule.

Appendix

The report was generated using the `Eisvogel.latex` template.

TO DO:

- WANDB run ids - HAND IN WANDB LOGS - READ OVER REPORT - INSERT THE LOSS CONVERGENCE COMPARISON CURVES